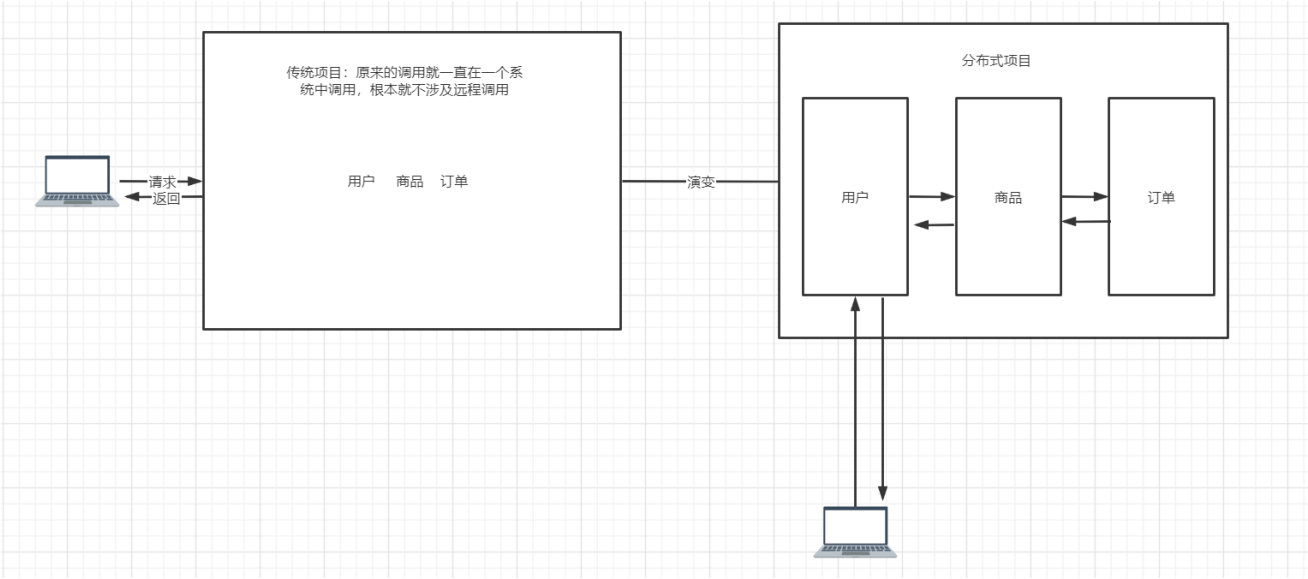
# RPC远程通信实战 Remote Procedure Call

- 什么是RPC
- 基于socket实现rpc
- 通过注解驱动来完成RPC的通信

首先我们知道在传统的单体应用下他的架构是这样的，画图




当出现分布式后，会发现远程通信就变的格外重要，也导致了出现很多这样的框架

# 如何实现远程通信

在我们现在应用层的远程通信有很多协议，比如：

http：

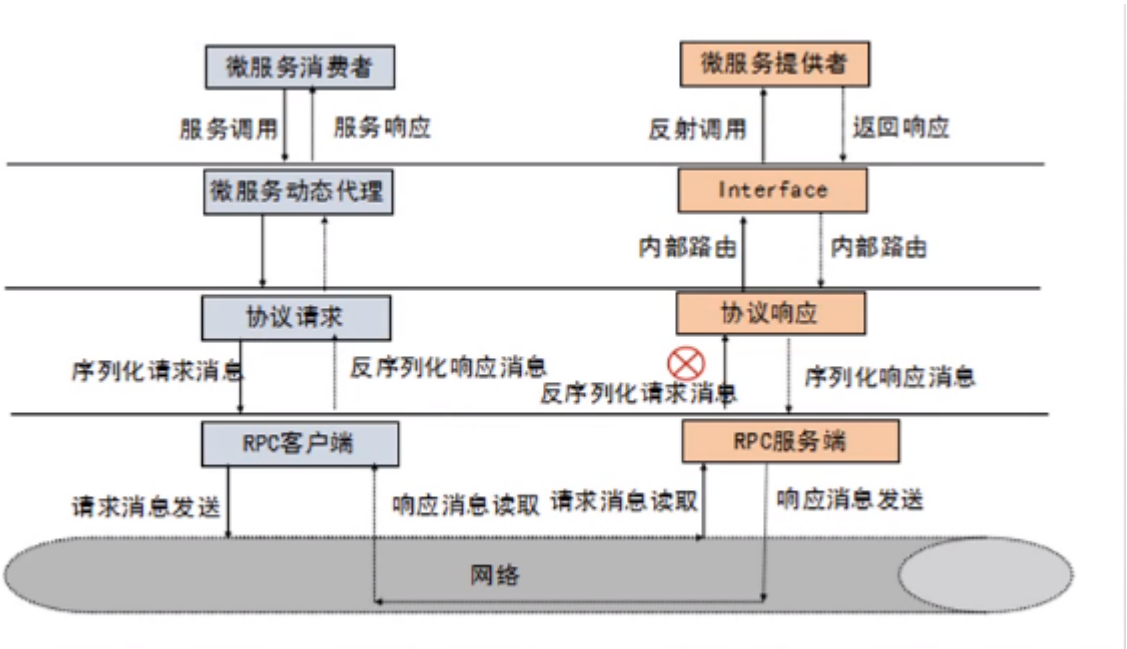
https:

Ftp:

ssh:

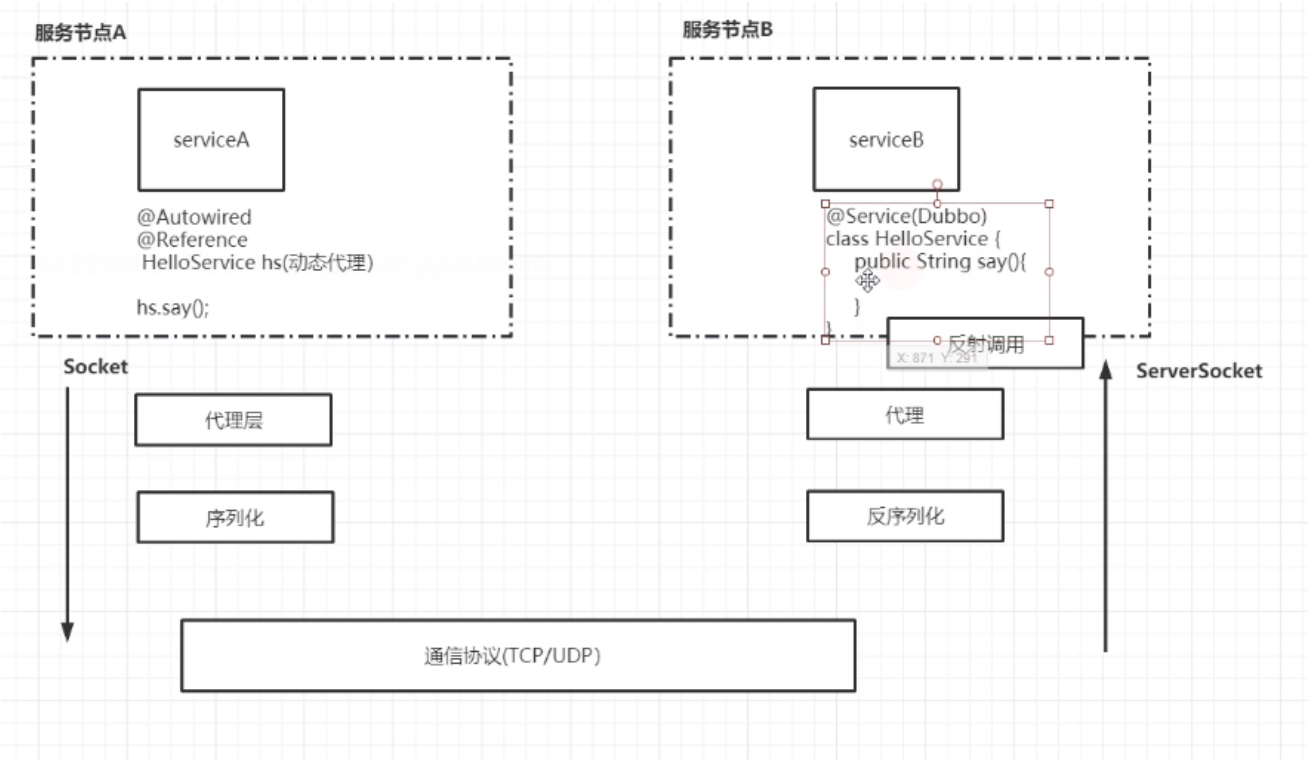
这些都是应用层的协议，那么我们服务与服务之间通信应该如何来实现，我们之前学习了socket套接字

- rpc框架去完成远程通信




所有的远程通信都是这样一个过程，可能只是实现方式不一样，比如使用的序列化不一样

所以，我们现在来自己手写一个rpc的代码，如何去实现呢，先给大家画个图




前面我们学习的socket编程，我们都是建立连接，然后传递数据，那么现在也是要这样去实现，但是现在我们不仅仅去传递数据，而且传递数据后要调用到对方的服务

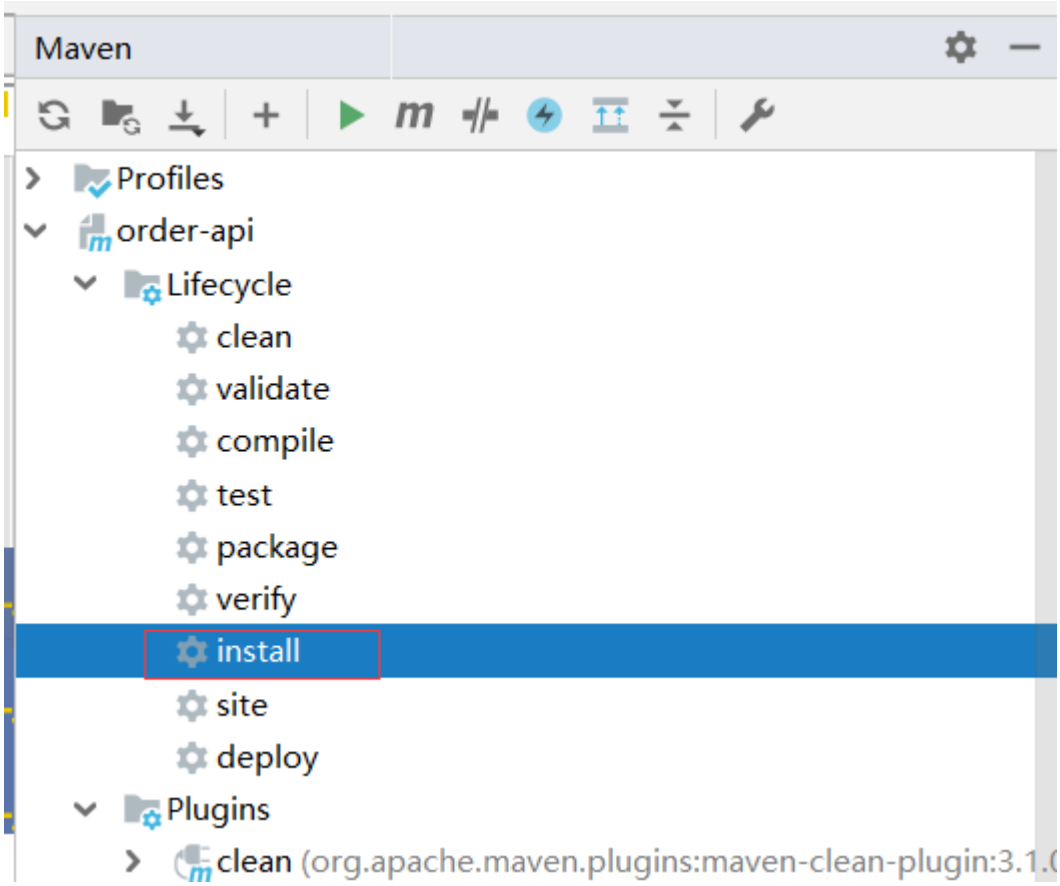
那么传递的数据就有要求了，因为要调用对方的服务，所以我们要把类，把方法名，参数等都要传输给对方，然后通过反射去调用

# 建立两个项目

思路：

- 先建立提供方的api的项目，将来注入到服务器上，供各个服务器使用（为什么要建立这个api的项目，是因为想要远程调用方法，必须要有他的类和方法名，我们建立api项目发布到私服上面去，然后再客户端引用，然后把类名和方法封装发送到服务端，服务端再去真正的实现这个接口，通过客户端传过来的类名和方法反射调用）
- 建立提供方的实现项目，真正的提供方，和客户端建立连接，然后得到数据，然后反射调用
- 建立客户端，然后引入api的项目，直接调用，这种调用肯定是不可能成功的，因为客户端这边都没有实现

> 在服务端order-api中定义接口类IOrderService，并定义两个方法
>
> ```
> public interface IOrderService {
>
> String queryOrderList();
>
> String orderById(String id);
> }
> ```
>
> 还有个东西，刚刚已经说了，客户端调用肯定是传输数据过来，然后反射调用，所以定义一个传输数据的类，并且序列化
>
> ```
> public class RpcRequest implements Serializable{
>
> private String className;// 类名称
> private String methodName;// 方法名称
> private Object[] args;// 参数
> private Class[] types;// 参数类型
> }
> ```
>
> 然后发布api




- 然后在客户端这边引入依赖

```
<dependency>
    <artifactId>order-api</artifactId>
    <groupId>com.gupaoedu.example</groupId>
    <version>1.0-SNAPSHOT</version>
</dependency>
```

> 开始调用服务端的IOrderService的实现类中的方法；
>
> 调用步骤：
>
> 1.首先要和服务端建立连接，用到Socket，需要端口和ip,生成代理类
>
> 2.在代理类中的处理类中封装数据RpcRequest，
>
> 3.最终调用服务端的方法，得到方法的返回值，并返回
>
> ```
> // 最后在客户端这边启动
> public class App
> {
> public static void main( String[] args ){
>  RpcProxyClient rpcProxyClient=new RpcProxyClient();
>  IOrderService
> iOrderService=rpcProxyClient.clientProxy(IOrderService.class,"localhost",8080);
>
>  System.out.println(iOrderService.orderById("11"));
>
>
>  }
> ```

```
}

// 建立一个生成代理类的类
public class RpcProxyClient {
    // 为什么这里的入参要是final,因为防止里面的逻辑把他们给改了，这里我们根据需求是不能让他们改的，这也是编程中的一个思想
public <T> T clientProxy(final Class<T> interfaceCls,final String host,final int port){
   return (T) Proxy.newProxyInstance(interfaceCls.getClassLoader(), new Class<?>[]{interfaceCls},new RemoteInvocationHandler(host,port));
}
}
```

```
// 然后在代理类中建立连接，并且封装数据并传输
public class RemoteInvocationHandler implements InvocationHandler{
private String host;
private int port;

public RemoteInvocationHandler(String host, int port) {
   this.host = host;
   this.port = port;
}

@Override
public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
   //先建立远程连接
   RpcNetTransport rpcNetTransport=new RpcNetTransport(host,port);
   //传递数据了?
   // 调用哪个接口、 哪个方法、方法的参数?
   RpcRequest request=new RpcRequest();
   request.setArgs(args);
   request.setClassName(method.getDeclaringClass().getName());
   request.setTypes(method.getParameterTypes()); //参数的类型
   request.setMethodName(method.getName());
   // 传输数据
   return rpcNetTransport.send(request);
}
}
```

```
// 真正传输数据
public class RpcNetTransport {

private String host;
private int port;

public RpcNetTransport(String host, int port) {
   this.host = host;
   this.port = port;
}


public Socket newSocket() throws IOException {
```

```
    Socket socket=new Socket(host,port);
    return socket;
  }

  public Object send(RpcRequest request){
    ObjectOutputStream outputStream=null;
    ObjectInputStream inputStream=null;
    try {
        Socket socket=newSocket();
        //IO操作
        outputStream=new ObjectOutputStream(socket.getOutputStream());
        outputStream.writeObject(request);
        outputStream.flush();
        inputStream=new ObjectInputStream(socket.getInputStream());
        return inputStream.readObject();
    } catch (IOException e) {
        e.printStackTrace();
    } catch (ClassNotFoundException e) {
        e.printStackTrace();
    } finally {
        //TODO 不写了
    }
    return null;
  }
  }
```

- 服务端接收数据，启动服务器
- 导入包

```
<dependency>
    <artifactId>order-api</artifactId>
    <groupId>com.gupaoedu.example</groupId>
    <version>1.0-SNAPSHOT</version>
</dependency>
```

- 实现IOrderService

```
public class OrderServiceImpl implements IOrderService {
    @Override
    public String queryOrderList() {
        return "queryOrderList";
    }

    @Override
    public String orderById(String s) {
        return "orderById";
    }
}
```

> 在这个地方首先我们要去接收客户端的请求，使用ServerSocket

> 我们使用线程池来帮助多个客户端连接过来
>
> 接收到数据后，进行反射调用，并把数据返回

```
// 需要被调用的类
IOrderService orderService=new OrderServiceImpl();
// 监听请求
RpcProxyServer rpcProxyServer=new RpcProxyServer();
rpcProxyServer.publisher(orderService,8080);
```

```
public class RpcProxyServer {
    // 定义一个线程池
private final ExecutorService executorService= Executors.newCachedThreadPool();
    // 然后开始监听客户端
public void publisher(Object service,int port){
   ServerSocket serverSocket=null;
   try {
       serverSocket=new ServerSocket(port);
       while(true){
           Socket socket=serverSocket.accept(); //监听客户端请求
           // 真正去调用
           executorService.execute(new ProcessorHandler(socket,service));
       }
   } catch (IOException e) {
       e.printStackTrace();
   }finally {
       if(serverSocket!=null){
           try {
               serverSocket.close();
           } catch (IOException e) {
               e.printStackTrace();
           }
       }
   }

}
}
```

请求到来后

```
public class ProcessorHandler implements Runnable{

private Socket socket;
private Object service;

public ProcessorHandler(Socket socket, Object service) {
   this.socket = socket;
   this.service = service;
}

@Override

public void run() {
```

```
    ObjectInputStream inputStream=null;
    ObjectOutputStream outputStream=null;
    try {
        // 得到输入流
        inputStream=new ObjectInputStream(socket.getInputStream());//?
        // 得到传输的数据并且反序列化
        RpcRequest request=(RpcRequest)inputStream.readObject();
        System.out.println("接收到客户端的数据=========" + request.toString());
        // 这个地方真正的通过反射去调用
        Object rs=invoke(request);
        System.out.println("服务端的执行结果: "+rs);
        //得到数据后并且回写客户端
        outputStream=new ObjectOutputStream(socket.getOutputStream());
        outputStream.writeObject(rs);
        outputStream.flush();
    } catch (Exception e) {
        e.printStackTrace();
    }finally {
        //TODO 关闭流
        if(inputStream!=null){
            try {
                inputStream.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
        if (outputStream!=null){
            try {
                outputStream.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}

private Object invoke(RpcRequest request) throws ClassNotFoundException,
NoSuchMethodException, InvocationTargetException, IllegalAccessException {
    //通过反射进行服务的调用
    Class clazz=Class.forName(request.getClassName());
    //找到目标方法
    Method method=clazz.getMethod(request.getMethodName(),request.getTypes());
    return method.invoke(service,request.getArgs());
}
}
```

- 那么这个有什么缺点没有

  在order-api中再添加一个接口类IUserService

```
public interface IUserService {

    public String findUserInfo(String name);
}
```

在order-service中实现接口类

```
public class UserService implements IUserService {
    @Override
    public String findUserInfo(String name) {
        return "my name is " + name;
    }
}
```

然后服务端启动， 此处发现如果有很多的service,我还要去不停的去修改代码去换，所以我们可以考虑把哪些api的实现类的bean给保存起来，调用的时候直接匹配拿到，然后反射调用，我们想到的就是在这些类上面可以加一个注解

```
public class App
{
    public static void main( String[] args )
    {

        IOrderService orderService = new OrderService();
        IUserService userService = new UserService();

        // 现在要去发布服务，并且建立监听
        RpcProxService rpcProxService = new RpcProxService();
        rpcProxService.publisher(orderService,8080);
    }
}
```

# 下面在服务端引入spring的依赖包，客户端改为springboot的项目

## 客户端改造

把项目改成springboot项目

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-dependencies</artifactId>
    <version>2.2.6.RELEASE</version>
</parent>

<dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>

</dependency>
```

注解：这个注解的意思就是，这个类是一个代理类，将来会去调用服务端的服务

```
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
@Component
public @interface GpReference {
}
```

建立一个Controller类

```
@RestController
public class TestController {

    @GpReference
    private IOrderService orderService; //

    @GetMapping("/test")
    public String test(){
        return orderService.queryOrderList();
    }

}
```

那么加了这个注解的，属性注入的时候要生成一个代理类，通过BeanPostProcessor对这个bean做一个改变

```
@Component
public class ReferenceInvokeProxy implements BeanPostProcessor{

    @Autowired
    RemoteInvocationHandler invocationHandler;

    @Override
    public Object postProcessBeforeInitialization(Object bean, String beanName) throws
BeansException {
        Field[] fields=bean.getClass().getDeclaredFields();
        for(Field field:fields){
            if(field.isAnnotationPresent(GpReference.class)){
                field.setAccessible(true);
                //针对这个加了GpReference注解的字段，设置为一个代理的值
                Object proxy= Proxy.newProxyInstance(field.getType().getClassLoader(),new
Class<?>[]{field.getType()},invocationHandler);
                try {
                    field.set(bean,proxy); //相当于针对加了GpReference的注解，设置了一个代理，这
个代理的实现是inovcationHandler
                } catch (IllegalAccessException e) {
                    e.printStackTrace();
                }
            }
        }
        return bean;
    }
```

```
}
```

添加一个配置文件application.properties

```
gp.host=localhost
gp.port=8888

spring.application.name=user-service
server.port=8080
```

- 然后在去改RemoteInvocationHandler,添加@Component

```
@Component
public class RemoteInvocationHandler implements InvocationHandler{
    @Value("${gp.host}")
    private String host;
    @Value("${gp.port}")
    private int port;

    public RemoteInvocationHandler() {
    }
```

我们的主类中

```
@SpringBootApplication
public class UserServiceMain {

    public static void main(String[] args) {
        SpringApplication.run(UserServiceMain.class,args);
    }
}
```

# 服务端改造

- 定义一个注解GpRemoteService

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Component
public @interface GpRemoteService {

}
```

- 配置spring的pom文件

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.2.7.RELEASE</version>
</dependency>
```

- 然后给实现类中添加注解GpRemoteService,添加这个注解的都会被注册到map中，后面直接去里面拿

```
@GpRemoteService
public class OrderServiceImpl implements IOrderService{
}
```

- 定义一个类实现BeanPostProcessor，实现postProcessAfterInitialization方法，在spring加载的时候，会在这里对bean做处理，最终把bean实例和方法封装成BeanMethod放入到map中

```
@Component
public class InitialMerdiator implements BeanPostProcessor{

    // 在bean加载之后执行
    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName) throws
BeansException {
        // 判断bean是否有GpRemoteService注解
        if(bean.getClass().isAnnotationPresent(GpRemoteService.class)){ //加了服务发布标
记的bean进行远程发布
            // 得到方法
            Method[] methods=bean.getClass().getDeclaredMethods();
            for(Method method:methods){
                // 通过接口的名字+方法名得到一个key和bean实例进行绑定放入到Mediator的map中
                String key=bean.getClass().getInterfaces()
[0].getName()+"."+method.getName();
                BeanMethod beanMethod=new BeanMethod();
                beanMethod.setBean(bean);
                beanMethod.setMethod(method);
                Mediator.map.put(key,beanMethod);
            }
        }
        return bean;
    }
}
```

- beanmethod类创建

```
public class BeanMethod {
    private Object bean;

    private Method method;

    public Object getBean() {
        return bean;

    }
```

```
    public void setBean(Object bean) {
        this.bean = bean;
    }

    public Method getMethod() {
        return method;
    }

    public void setMethod(Method method) {
        this.method = method;
    }
}
```

```
public class Mediator {

    //用来存储发布的服务的实例(服务调用的路由)
    public static Map<String ,BeanMethod> map=new ConcurrentHashMap<>();

    private volatile static Mediator instance;

    private Mediator(){};
  // 单例
    public static Mediator getInstance(){
        if(instance==null){
            synchronized (Mediator.class){
                if(instance==null){
                    instance=new Mediator();
                }
            }
        }
        return instance;
    }

}
```

- 然后在容器启动完成后，发布一个事件，这个就是观察者模式，实现ApplicationListener，重写onApplicationEvent就可以了

```
@Component
public class SocketServerInitial implements
ApplicationListener<ContextRefreshedEvent>{
    // 定义一个线程池
    private final ExecutorService executorService= Executors.newCachedThreadPool();

    // 在应用事件上
    @Override

    public void onApplicationEvent(ContextRefreshedEvent contextRefreshedEvent) {
```

```
        //启动服务
        ServerSocket serverSocket=null;
        try {
            serverSocket=new ServerSocket(8888);
            while(true){
                Socket socket=serverSocket.accept(); //监听客户端请求
                // 使用线程池执行任务,此时server将不用去传递了
                executorService.execute(new ProcessorHandler(socket));
            }
        } catch (IOException e) {
            e.printStackTrace();
        }finally {
            if(serverSocket!=null){
                try {
                    serverSocket.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        }
    }
}
```

- 然后在定义一个线程ProcessorHandler实现Runnable接口

```
public class ProcessorHandler implements Runnable{

    private Socket socket;

    public ProcessorHandler(Socket socket) {
        this.socket = socket;
    }

    @Override
    public void run() {
        ObjectInputStream inputStream=null;
        ObjectOutputStream outputStream=null;
        try {
            inputStream=new ObjectInputStream(socket.getInputStream());//?
            // 得到客户端的数据反序列化
            RpcRequest request=(RpcRequest)inputStream.readObject(); //反序列化
            // 得到一个单例对象，去处理客户端来的数据
            Mediator mediator=Mediator.getInstance();
            // 这个里面已经通过客户端的数据，通过反射已经调用了数据，然后给客户端返回
            Object rs=mediator.processor(request);
            System.out.println("服务端的执行结果: "+rs);
            // 写回客户端
            outputStream=new ObjectOutputStream(socket.getOutputStream());
            outputStream.writeObject(rs);
            outputStream.flush();
        } catch (Exception e) {
            e.printStackTrace();
        }finally {
```

```
            //TODO 关闭流
            if(inputStream!=null){
                try {
                    inputStream.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
            if (outputStream!=null){
                try {
                    outputStream.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        }
    }
}
```

- 在Mediator类中添加方法processor，从map中取出BeanMethod,可以得到bean和method，然后通过反射调用

```
    // key是名字+方法，从map中取出BeanMethod,可以得到bean和method，然后通过反射调用
    public Object processor(RpcRequest request){
        String key=request.getClassName()+"."+request.getMethodName(); //key
        BeanMethod beanMethod=map.get(key);
        if(beanMethod==null){
            return null;
        }
        Object bean=beanMethod.getBean();
        Method method=beanMethod.getMethod();
        try {
            return method.invoke(bean,request.getArgs());
        } catch (IllegalAccessException e) {
            e.printStackTrace();
        } catch (InvocationTargetException e) {
            e.printStackTrace();
        }
        return null;
    }
```

```
@Configuration
@ComponentScan("com.gupaoedu.ssvip")
public class Bootstrap {

    public static void main(String[] args) {
        ApplicationContext applicationContext=
                new AnnotationConfigApplicationContext(Bootstrap.class);
    }
}
```

# 最后把客户端的代码抽取一个公共的服务

- 在服务端创建一个项目gp-util,把工具类复制过来,导入pom文件

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.2.7.RELEASE</version>
</dependency>

<dependency>
    <artifactId>order-api</artifactId>
    <groupId>com.gupaoedu.example</groupId>
    <version>1.0-SNAPSHOT</version>
</dependency>
```

- 删除这些类,导入pom文件

```
<dependency>
    <groupId>com.cn</groupId>
    <artifactId>gp-wentai-util</artifactId>
    <version>1.0-SNAPSHOT</version>
</dependency>
```

- 在启动类上面添加扫描包路径

```
@SpringBootApplication
@ComponentScan("com.cn")
public class UserServiceMain {

    public static void main(String[] args) {
        SpringApplication.run(UserServiceMain.class,args);
    }
}
```

- 配置resources目录跟java目录平级，然后再建立META-INF，新建文件spring.factories,添加以下内容

```
org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
com.cn.ReferenceInvokeProxy,\
com.cn.RemoteInvocationHandler
```